

UrbanPulse Architecture

MetroConnect Real-Time Urban Operations Intelligence Platform

Architecture Choice: Lambda Hybrid

UrbanPulse uses a **Lambda Architecture** with Flink for the speed layer (sub-2-minute AQI alerts, signal control) and Spark for the batch layer (15-minute ward aggregations, councillor reports). A pure Kappa approach was rejected because government reporting mandates require separate historical reprocessing with auditable batch runs — not streaming recomputation. The two layers share the same Kafka topics so there is no duplicate ingestion code.

LAYER 1 — DATA SOURCES



BUS GPS

12,000 BMTC/BEST buses
GPS tracker — every 400ms

~2,400 events/sec



TRAFFIC SIGNALS

3,800 junction sensors
vehicle count + wait time

~380 events/sec



AIR QUALITY

600 AQI monitors
PM2.5 · PM10 · NO₂ · AQI

~60 events/sec



SMART METERS

1.1 M electricity meters
kWh · voltage · power factor

~1,100 events/sec



LAYER 2 — KAFKA INGESTION (3-BROKER CLUSTER)

APACHE KAFKA — 3 BROKERS · DOCKER COMPOSE

urbanpulse.bus_gps

40 partitions · key = route_id
Retention: 24 hours

urbanpulse.traffic_signals

15 partitions · key = junction_id
Retention: 7 days

urbanpulse.air_quality

10 partitions · key = sensor_id
Retention: 90 days

urbanpulse.smart_meters

20 partitions · key = meter_id
Retention: 365 days

urbanpulse.incidents

5 partitions · Flink alert output

urbanpulse.dlq

3 partitions · Dead-letter queue

urbanpulse.health_advisories

5 partitions · Spark SQL output

urbanpulse.ward_energy_summary

10 partitions · Spark output



LAYER 3 — PROCESSING LAYER (LAMBDA HYBRID)



REAL-TIME — APACHE FLINK (PYFLINK · LOCAL MODE)

- ▶ **AQI Emergency** — keyed state · AQI > 300 → alert ≤ 2 min
- ▶ **Traffic Gridlock** — avg wait > 180s for 3 cycles
- ▶ **Bus Bunching** — 2 buses same route < 200m for > 5 min
- ▶ **Event-time watermarks** — handles out-of-order arrival
- ▶ **Bus GPS Enrichment** — join GPS stream with route_schedule
- ▶ Output → `urbanpulse.incidents`



BATCH + STREAMING — APACHE SPARK (PYSARK · LOCAL[*])

- ▶ **Ward Energy** — 15-min tumbling window · total kWh, avg PF
- ▶ **Dual output** — Kafka topic + Parquet (ward_id, date)
- ▶ **Health Advisory SQL** — 10-min rolling avg AQI per zone
- ▶ **Zone join** — static zone_profile table (population, schools)
- ▶ **Filter** rolling_avg_aqi > 150 → Update mode output
- ▶ Output → `urbanpulse.health_advisories` + Parquet



LAYER 4 — STORAGE LAYER (POLYGLOT PERSISTENCE)

InfluxDB

Time-series sensor data
AQI · traffic signals · voltage

Purpose-built TSDB; nanosecond timestamps; retention policies

PostGIS (PostgreSQL)

Geospatial bus positions
lat/lon, route shapes, stops

Native geo-indexing (GiST); ST_Distance for bunching

Apache Parquet

Historical AQI records
Batch Spark output; councillor reports

Columnar; partitioned by zone + date; open-source

PostgreSQL

Councillor report aggregates
weekly/monthly ward summaries

ACID; easy SQL reporting; zero license cost



LAYER 5 — SERVING LAYER

Grafana Dashboards

Live AQI heatmaps · bus fleet map · ward energy tiles
For ward officers — no SQL needed

FastAPI Advisory API

REST endpoint for health advisories
Pulls from health_advisories topic

Signal Control Interface

Adaptive signal timing
HIGH_PRIORITY consumer → MQTT

Councillor Reports

Weekly/monthly PDF from Parquet
PostgreSQL aggregates → PDF

● Data Sources

● Kafka Ingestion

● Flink Real-Time

● Spark Batch

● Storage

● Serving